

Исследовательская лабораторная работа №1.

Как создать, собрать и запустить программу?

Дисциплина: Архитектура ЭВМ

Инструменты: .NET SDK, VS Code, Git, терминал (Git Bash или PowerShell)

Это первая **исследовательская** лабораторная работа курса. Формат отличается от практических работ, которые вы делали раньше: здесь нет пошаговой инструкции с готовым ответом в конце. Вместо этого — цикл, который будет повторяться во всех дальнейших исследовательских работах:

Я предполагаю... → Я проверяю... → Я наблюдаю... →

Я сравниваю... → Я объясняю... → Я проверяю объяснение... → Я делаю

вывод...

Правильного ответа "заранее" в этой работе нет — есть эксперимент и то, что вы в нём обнаружите.

Исследовательский вопрос

На прошлом занятии вы могли заметить: второй запуск **dotnet run** прошёл заметно быстрее первого. Вы также видели, что после **dotnet run** на диске появляются папки **bin** и **obj**, но не разбирали, что конкретно происходит внутри них.

Сегодняшний вопрос: **из каких отдельных шагов состоит путь от исходного кода до запущенной программы, и какие из этих шагов компьютер умеет пропускать при повторном запуске?**

Цель

- Экспериментально разделить то, что обычно выполняется одной командой (**dotnet run**), на отдельные шаги: сборку и запуск.
- Измерить время выполнения этих шагов в разных условиях и найти закономерность.
- Самостоятельно разобраться, что хранится в папках **obj** и **bin**, и связать это с результатами измерений.
- Сформулировать собственное объяснение — а не зафиксировать чужое.

Краткая теория

Команда **dotnet run**, которой вы пользовались, на самом деле выполняет за вас сразу несколько операций. Две из них можно вызвать отдельно:

dotnet build # собрать программу, ничего не запуская

dotnet run # собрать (если нужно) и сразу запустить

Больше в теории сегодня рассказывать не будем — весь смысл занятия в том, чтобы вы сами, через эксперимент, обнаружили, чем **build** отличается от **run**, и почему одна и та же команда иногда выполняется мгновенно, а иногда — заметно дольше.

Как измерять время выполнения команды

time dotnet build

Вывод будет выглядеть примерно так:

```
real    0m2.302s
user    0m0.030s
sys     0m0.015s
```

Вам понадобится строка **real** — реальное время от старта команды до её завершения.

Гипотеза

Прежде чем начать эксперимент, создайте в папке с вашим проектом файл **REPORT.md** и запишите в него свой прогноз, письменно ответив на два вопроса:

1. Как вы думаете, **dotnet build** каждый раз пересобирает программу заново «с нуля», или может «вспомнить» предыдущую сборку и что-то переиспользовать?
2. Если код никак не менялся с прошлого запуска, будет ли повторный **dotnet build** таким же по времени, как первый, быстрее или медленнее? Обоснуйте предположение в одном-двух предложениях.

Важно зафиксировать именно то, что вы думаете *сейчас*, до эксперимента — дальше вы будете сверяться с этой записью.

Часть 1. Базовые измерения

Полностью удалите результаты предыдущих сборок, чтобы начать с чистого состояния:

cd Lab01

rm -rf bin obj

Замерьте время первой (чистой) сборки и запишите значение в таблицу в **REPORT.md**:

time dotnet build

Запуск	Условие	Время
1	чистая сборка (bin/obj удалены)	

Теперь, **ничего не меняя в коде**, выполните точно ту же команду ещё раз и запишите время под номером 2:

time dotnet build

Затем откройте **Program.cs**, добавьте одну новую строку (например, ещё один `Console.WriteLine`), сохраните файл и снова замерьте время — запишите под номером 3:

time dotnet build

Запуск	Условие	Время
1	чистая сборка (bin/obj удалены)	
2	повторная сборка, код не менялся	
3	сборка после изменения одной строки	

Сравните три значения. Как изменение одного файла повлияло на продолжительность сборки по сравнению с запуском 1 и запуском 2? Запишите короткое наблюдение в **REPORT.md** — пока без объяснения, просто что вы увидели. (Абсолютные секунды у вас и у соседа по парте могут отличаться — компьютеры разные. Важно не число само по себе, а соотношение между запусками.)

Часть 2. Что находится внутри obj и bin

Перед вами два каталога, которые появились после сборки. Посмотрите на содержимое каждого:

ls -la obj/Debug/net*/

ls -la bin/Debug/net*/

Сравните списки файлов. Не пытайтесь сразу найти "правильный" ответ в интернете — для начала запишите в **REPORT.md** собственное предположение:

- Чем, на ваш взгляд, **obj** отличается от **bin** по назначению?
- Какие файлы в каждой папке кажутся вам наиболее важными и почему?

Затем найдите в **obj** файл с расширением `.cache` (например, что-то вроде `Lab01.csproj.CoreCompileInputs.cache`) и откройте его любым текстовым редактором.

Наблюдение для отчёта: что хранится внутри этого файла — текст программы, дата, число, что-то похожее на хеш-сумму? Как вы думаете, зачем компилятору может понадобиться такой файл между запусками сборки?

Часть 3. Эксперимент: что будет, если удалить только часть

Сейчас вы проверите свою догадку из Части 2 на практике. Выполните три сборки подряд, каждый раз меняя только то, что удалено перед сборкой.

Состояние А. `bin` и `obj` уже существуют (после Части 1 и 2) — просто соберите ещё раз:

`time dotnet build`

Состояние В. Удалите только `bin`:

`rm -rf bin`

`time dotnet build`

Состояние С. Удалите и `bin`, и `obj`:

`rm -rf bin obj`

`time dotnet build`

Запишите время всех трёх сборок в отдельную таблицу в **REPORT.md**:

Состояние	Что удалено	Время
А	ничего	
В	только <code>bin</code>	
С	<code>bin</code> и <code>obj</code>	

Сравните состояния В и С. Ответьте письменно:

Почему удаление `bin` и удаление `obj` приводят к разному поведению сборки?

Совпадает ли это с вашим предположением о назначении `obj` и `bin`, которое вы записали в Части 2? Если нет — поправьте своё объяснение с учётом того, что увидели теперь.

Часть 4. Файл проекта

Откройте файл **Lab01.csproj**, приблизительный вывод:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    ...
  </PropertyGroup>
</Project>
```

Это не код на C# — конфигурация проекта в формате XML. Найдите в нём и запишите в **REPORT.md**:

- тип приложения (какое значение стоит в OutputType);
- целевую версию .NET;
- любые другие параметры, которые вы можете распознать или предположить, за что они отвечают.

Не пытайтесь разобрать все возможные теги — достаточно того, что вы узнали.

Ответьте письменно на вопрос:

Почему файл проекта нужен системе сборки, если весь исходный код находится в Program.cs?

Часть 5. Объясняем результат

Соберите всё, что вы увидели за занятие: разницу во времени между запусками 1–3, разницу между состояниями A–C, содержимое .cache-файла. На основе этого сформулируйте письменный ответ на вопрос:

Почему вторая сборка без изменения исходного кода может оказаться значительно быстрее первой?

Опишите объяснение своими словами, опираясь на то, что вы наблюдали, а не на то, что "где-то слышали". После того как вы запишете собственную версию, можете свериться с термином: то, что вы, скорее всего, только что описали, называется **инкрементальной сборкой** — способностью системы сборки повторно использовать уже готовый результат вместо полной пересборки, если она может убедиться, что ничего не изменилось.

Если запуск 3 (после изменения одной строки) в вашем случае оказался почти таким же быстрым, как запуск 2 — это тоже стоит объяснить. Свяжите это с тем, что вы выяснили о .csproj и obj: что именно системе сборки нужно было пересобрать заново, а что — нет?

Что включить в REPORT.md

Отчёт короткий, но должен показывать ход мысли, а не просто список команд:

1. Гипотеза (записана до эксперимента).
2. Таблица измерений (Часть 1) и таблица эксперимента с удалением (Часть 3).
3. Наблюдения по obj и bin, включая содержимое .cache-файла.
4. Ответ на вопрос "почему удаление bin и obj ведёт себя по-разному".

5. Собственное объяснение инкрементальной сборки (Часть 5).
6. Один абзац: совпал ли итоговый результат с первоначальной гипотезой, и если нет — в чём вы ошиблись.

Зафиксируйте работу в Git:

git add .

git commit -m "Lab01: build vs run investigation"

git push

Контрольные вопросы

1. Почему второй dotnet build может быть быстрее первого?
2. Почему удаление bin и удаление obj — не одно и то же с точки зрения последствий для сборки?
3. Что изменится, если bin и obj не сохранять на диске между запусками?
4. Зачем системе сборки нужен .csproj, если весь код — в Program.cs?
5. Почему изменение одного исходного файла не обязательно означает полную пересборку всего проекта?
6. Почему bin и obj обычно не хранят в Git (вспомните .gitignore из прошлого занятия)?

Дополнительное задание ★★

Вы исследовали, как ведёт себя **dotnet build**. А распространяется ли то же самое ускорение на dotnet run целиком, или только на часть его работы?

rm -rf bin obj

time dotnet run # чистый запуск

time dotnet run # запуск без изменений

Сравните разницу времени здесь с разницей, которую вы получили для dotnet build в Части 1. Она примерно такая же или заметно другая? Если время dotnet run во втором запуске сократилось меньше, чем время dotnet build — предположите, на что ещё, кроме сборки, могло уйти это дополнительное время. Запишите наблюдение и предположение в **REPORT.md** отдельным пунктом — точный ответ разбирать пока не будем, вы вернётесь к этому вопросу позже в курсе.

git add .

git commit -m "Lab01: extra task - build vs run timing"

git push